Vidyalankar Institute of Technology

Department of Computer Engineering

DEVOPS MINI PROJECT

Week 15 Report

Automated Grocery Inventory Dashboard

Name:	Prachi Patil
Roll No.:	23102B0038
Class:	BE - VII
Division:	B
Academic Year:	2026–27

Abstract

The Automated Grocery Inventory Dashboard is a DevOps-enabled web application designed to manage grocery inventory items, stock transactions, search and filtering, inventory status, and exception alerts.

The project was developed incrementally using a complete DevOps workflow involving Git and GitHub for version control, Jenkins for continuous integration and deployment automation, Selenium for automated functional testing, Maven for build and packaging, Docker for containerization, a local Docker registry for image distribution, and Ansible for configuration management, provisioning, deployment, and reliability validation.

Week 15 represents the final end-to-end release phase of the project. The complete workflow was integrated and executed from source-code checkout through Selenium quality validation, WAR packaging, Docker image creation, Docker registry push, Ansible provisioning, and final HTTP health validation.

The final Jenkins Build #3 completed successfully. All six Selenium tests passed with zero failures and zero errors. The application was packaged as a WAR, containerized into the versioned Docker image `localhost:5001/grocery-inventory-dashboard:3`, deployed through Ansible, and validated through an HTTP health check on port 8086.

The final release was merged into the `develop` branch through Pull Request #9 and tagged as `v1.0.0`.

1. Introduction

Week 15 is the final stage of the Automated Grocery Inventory Dashboard DevOps project. The objective of this week was to demonstrate the complete software delivery lifecycle as one integrated workflow rather than as independent tools.

The final workflow combines:

- Git and GitHub for source-code management.
- Jenkins for automated CI/CD orchestration.
- Maven for application compilation, testing, and WAR packaging.
- Selenium for automated functional testing.
- Docker for application containerization.
- A local Docker registry for image storage and distribution.
- Ansible for provisioning and deployment.
- HTTP health validation for final reliability verification.

The Week 15 implementation was developed on the branch:

`feature/final-end-to-end-release`

After validation, the branch was merged into `develop` through Pull Request #9.
The final release was tagged:

`v1.0.0`

2. Objectives

The major objectives of Week 15 were:

1. Integrate the complete DevOps workflow into a single Jenkins pipeline.
2. Execute Selenium automated tests as a quality gate.
3. Prevent packaging and deployment when the quality gate fails.
4. Package the application as a deployable WAR artifact.

5. Build and version a Docker image automatically.
6. Push the Docker image to the local Docker registry.
7. Provision and deploy the final release using Ansible.
8. Validate idempotent Ansible behaviour.
9. Perform a final HTTP health check.
10. Document troubleshooting and limitations.
11. Merge the final implementation into `develop`.
12. Create the final `v1.0.0` release tag.

3. Final Technology Stack

Table 1: Final technology stack

Layer	Technology
Programming Language	Java 21
Framework	Spring Boot 4.1.0
Build Tool	Maven
Frontend	Thymeleaf
Database	MySQL
Version Control	Git
Repository	GitHub
CI/CD	Jenkins
Functional Testing	Selenium WebDriver + JUnit 5
Application Packaging	WAR
Application Server	Apache Tomcat 10.1.x
Containerization	Docker
Container Registry	Local Docker Registry
Configuration Management	Ansible 2.21.4
Operating Environment	macOS on Apple Silicon

4. Final DevOps Architecture

The final architecture connects source control, automated testing, application packaging, containerization, registry management, provisioning, and deployment.

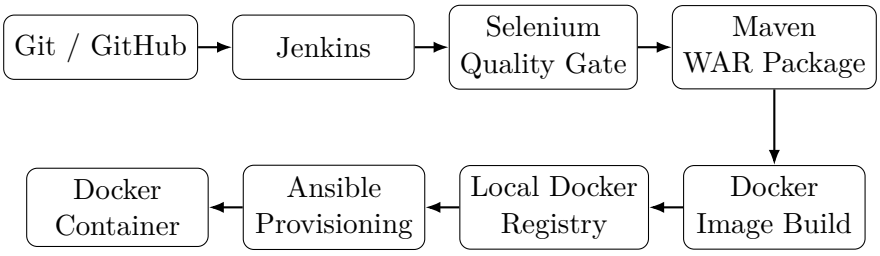


Figure 1: Final end-to-end DevOps architecture

The workflow follows the sequence:

Git Commit → Jenkins → Selenium → Maven → Docker → Registry → Ansible → Container → Health Check

5. Final Git Release

The final Week 15 development was performed on:

`feature/final-end-to-end-release`

The branch contained three Week 15 implementation commits:

Table 2: Week 15 implementation commits

Commit	Description
`1347542`	Integrate final end-to-end DevOps pipeline
`42d9d5e`	Fix Selenium stale element handling
`0935cf9`	Restore Dockerfile for final release

The final branch was merged into `develop` through Pull Request #9.
The merge commit was:

`81afb33`

The final release tag was created using:

```
git tag -a v1.0.0 -m "Final Week 15 End-to-End DevOps Release"
git push origin v1.0.0
```

The final repository contains the release tags:

`v0.6.0, v0.13.0, v0.14.0, v1.0.0`

6. Final Jenkins Pipeline

The final Jenkins job was:

Automated-Grocery-Inventory-Final-Release

The pipeline performs the following stages:

1. Checkout
2. Selenium Quality Gate
3. Package WAR
4. Build Docker Image
5. Push Docker Image
6. Ansible Provision and Deploy
7. Final Health Check

The pipeline uses the Jenkins build number as the Docker image version.
For Build #3:

`BUILD_NUMBER = 3`

Therefore, the generated image was:

`localhost:5001/grocery-inventory-dashboard:3`

This provides a direct relationship between the Jenkins build and the Docker image version.

7. Selenium Quality Gate

Selenium testing was placed before application packaging and deployment.

The final Selenium suite contains six automated tests covering the critical inventory workflows.

Table 3: Final Selenium validation

No.	Test Area	Validation
1	Inventory View	Verifies that the inventory dashboard loads correctly.
2	Add Item	Creates a new inventory item and verifies its presence.
3	Search Item	Searches for an inventory item and verifies the result.
4	Stock Transaction	Validates stock-in and stock-out behaviour.
5	Exception Alerts	Verifies inventory exception alert functionality.
6	Additional Inventory Journey	Validates another critical inventory workflow in the automated suite.

The final Jenkins run reported:

6 tests executed
0 failures
0 errors
0 skipped

The Selenium stage therefore acted as the quality gate for the subsequent packaging and deployment stages.

8. Selenium Issue and Correction

During the Week 15 pipeline execution, the first Jenkins build encountered a Selenium stale element error in the search test.

The problem occurred because the test attempted to retain a WebElement while the page content could be refreshed or replaced.

The test was changed to repeatedly locate the element during the explicit wait and handle a possible stale element exception.

The final approach was:

```
boolean resultDisplayed = wait.until(webDriver -> {
    try {
        return webDriver.findElement(riceResult).isDisplayed();
    } catch (StaleElementReferenceException e) {
        return false;
    }
});
```

The corrected test was executed locally and subsequently passed in Jenkins.

This demonstrates that a failed quality gate was investigated and corrected before the final release.

9. WAR Packaging

After the Selenium quality gate passed, Jenkins packaged the application as a WAR file using Maven.

The packaging command was:

```
./mvnw package -DskipTests
```

The generated artifact was:

`target/automated-grocery-inventory-dashboard-0.0.1-SNAPSHOT.war`

The final WAR generated during Build #3 was approximately 51 MB.
The WAR is used as the application artifact copied into the Tomcat-based Docker image.

10. Docker Image Build

The final Docker image is based on:

`tomcat:10.1-jdk21-temurin`

The Dockerfile removes the default Tomcat applications and copies the generated WAR into the Tomcat deployment directory.
The final Dockerfile structure is:

```
FROM tomcat:10.1-jdk21-temurin

RUN rm -rf /usr/local/tomcat/webapps/*

COPY target/automated-grocery-inventory-dashboard-0.0.1-SNAPSHOT.war \
     /usr/local/tomcat/webapps/automated-grocery-inventory-dashboard.war

EXPOSE 8080

CMD ["catalina.sh", "run"]
```

For Jenkins Build #3, the image was:

`localhost:5001/grocery-inventory-dashboard:3`

The image was successfully built and pushed to the local registry.

11. Docker Registry

A local Docker registry was used to store and distribute the application image.
The registry was configured using:

```
docker run -d \
  --name grocery-local-registry \
  -p 5001:5000 \
  --restart=always \
  registry:2
```

The external registry endpoint was:

`localhost:5001`

The final image was pushed using:

```
docker push localhost:5001/grocery-inventory-dashboard:3
```

The successful push generated the image digest:

`sha256:e0d8659cbee42408d7f5501636ff8bb885e681785fca242d562c0515d5af495f`

12. Ansible Provisioning

Ansible was selected for configuration management and final provisioning.

The final Week 15 playbook is:

`ansible/week15/final-provision.yml`

The playbook performs:

1. Target information collection.
2. Deployment directory validation.
3. Docker availability verification.
4. Existing container inspection.
5. Existing image inspection.
6. Existing container status inspection.
7. Deployment decision.
8. Removal of an outdated or stopped container when required.
9. Pulling the desired Docker image.
10. Starting the desired container.
11. Waiting for application startup.
12. HTTP health validation.
13. Final container status validation.
14. Final image validation.

The final application container was:

`grocery-inventory-week15`

The external application port was:

`8086`

The Docker container internally exposes port 8080.

Therefore:

$$8086_{\text{host}} \rightarrow 8080_{\text{container}}$$

13. Idempotency Validation

An important Week 15 reliability requirement was to verify that repeated execution of the Ansible playbook does not unnecessarily recreate an already-correct deployment.

During the final pipeline execution, the existing deployment was inspected.

The playbook detected:

- Existing container: Yes
- Existing image: `localhost:5001/grocery-inventory-dashboard:3`
- Existing status: `running`
- Desired image: `localhost:5001/grocery-inventory-dashboard:3`

Therefore:

Deployment required = False

The final Ansible recap was:

```
ok=13 changed=0 failed=0 skipped=4
```

This demonstrates idempotent behaviour because no unnecessary deployment change was performed when the desired state was already satisfied.

14. Final Application Health Check

The final deployed application was validated through:

```
http://localhost:8086/automated-grocery-inventory-dashboard/items
```

The Ansible playbook used an HTTP GET request and expected status code 200.
The final Jenkins health check also used:

```
curl --fail --silent --show-error \
     --retry 5 \
     --retry-delay 5 \
     --output /dev/null \
     "http://localhost:8086/automated-grocery-inventory-dashboard/items"
```

The final result was:

HTTP Health Check: PASSED

The application was therefore successfully deployed and responding.

15. Final End-to-End Pipeline Result

The final Jenkins Build #3 completed the entire workflow successfully.

Table 4: Final Jenkins Build #3 result

Stage	Result
Checkout	SUCCESS
Selenium Quality Gate	SUCCESS
WAR Packaging	SUCCESS
Docker Image Build	SUCCESS
Docker Image Push	SUCCESS
Ansible Provision and Deploy	SUCCESS
Final Health Check	SUCCESS
Overall Pipeline	SUCCESS

The final release information was:

Table 5: Final release information

Parameter	Value
Jenkins Job	Automated-Grocery-Inventory-Final-Release
Jenkins Build	#3
Git Branch	develop after merge
Merge Commit	`81afb33`
Release Tag	`v1.0.0`
Docker Image	`localhost:5001/grocery-inventory-dashboard:3`
Container	`grocery-inventory-week15`
Application Port	8086
Container Port	8080
Selenium Tests	6
Test Failures	0
Test Errors	0
Ansible Changes	0 on final idempotent run
HTTP Status	200

16. Troubleshooting and Corrective Actions

The final development process involved resolving actual integration issues before completing the release.

16.1 Selenium Stale Element Error

The first Week 15 Jenkins execution encountered a stale element error in the search test.

Cause: The search result element could become stale while the page was being updated.

Resolution: The Selenium wait was changed to locate the element again inside the wait condition and handle `StaleElementReferenceException`.

Result: The final Selenium suite passed all six tests.

16.2 Missing Dockerfile

The second Jenkins execution passed the Selenium stage but failed during Docker image creation because the final branch did not contain the root-level Dockerfile.

Cause: The Dockerfile used during the earlier Docker week had not been present in the Week 15 branch.

Resolution: The established project Dockerfile was restored at the project root.

Result: Docker image construction and subsequent stages completed successfully in Build #3.

16.3 Ansible Idempotency

The final deployment did not recreate the application container unnecessarily.

Cause: The existing container already matched the desired image and was running.

Resolution: No deployment action was required.

Result:

```
ok=13 changed=0 failed=0 skipped=4
```

17. Technical Documentation

The final project documentation covers the complete DevOps workflow.

The documented areas include:

- Project overview.

- Application architecture.
- Git branching workflow.
- Jenkins CI/CD pipeline.
- Selenium test automation.
- Maven WAR packaging.
- Docker image construction.
- Local Docker registry.
- Ansible configuration management.
- Automated provisioning.
- Health checks.
- Idempotency.
- Troubleshooting procedures.
- Known limitations.
- Future enhancements.

18. Limitations

The final system has several limitations:

1. The Docker registry is local rather than a cloud-hosted registry.
2. The deployment environment is a local development machine.
3. The MySQL database is externally available through the host machine.
4. Database credentials are configured for the local development environment.
5. The final deployment uses a single application container.
6. Jenkins is running locally.
7. The application does not currently use an external production-grade monitoring platform.
8. The system does not currently implement automated rollback to a previous image version.
9. The final health check validates HTTP availability but does not perform extensive application-level monitoring.

19. Future Enhancements

The following enhancements can improve the project further:

1. Deploy the Docker registry to a cloud environment.
2. Use a managed MySQL database for production deployment.
3. Introduce Docker Compose or Kubernetes for multi-container deployment.
4. Add automated database backup and recovery.
5. Introduce Prometheus and Grafana for monitoring.
6. Add centralized logging.
7. Implement automated rollback to a previous Docker image.
8. Add security scanning to the Jenkins pipeline.
9. Add dependency vulnerability scanning.
10. Introduce environment-specific configuration and secrets management.
11. Add automated release notifications.
12. Expand Selenium coverage for additional user journeys.

20. Final Evidence

The following screenshots should be included as evidence of the final release.

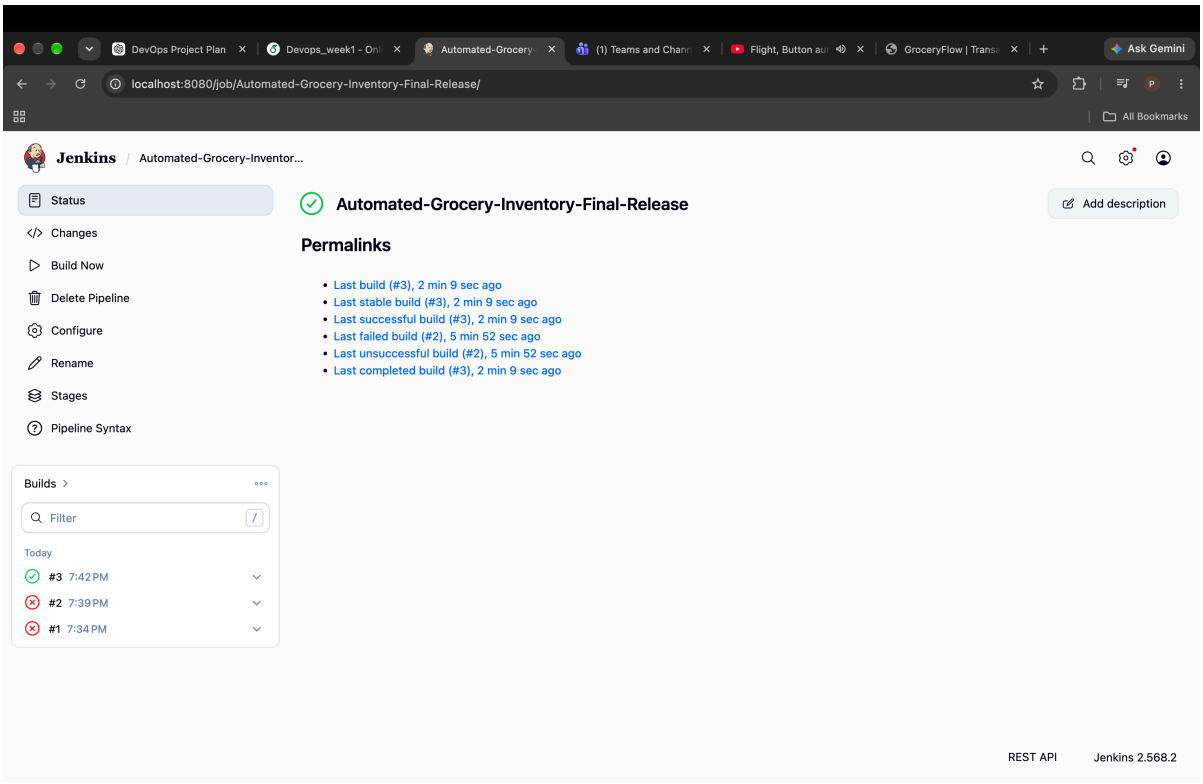


Figure 2: Jenkins Build #3 showing successful completion of the end-to-end pipeline.

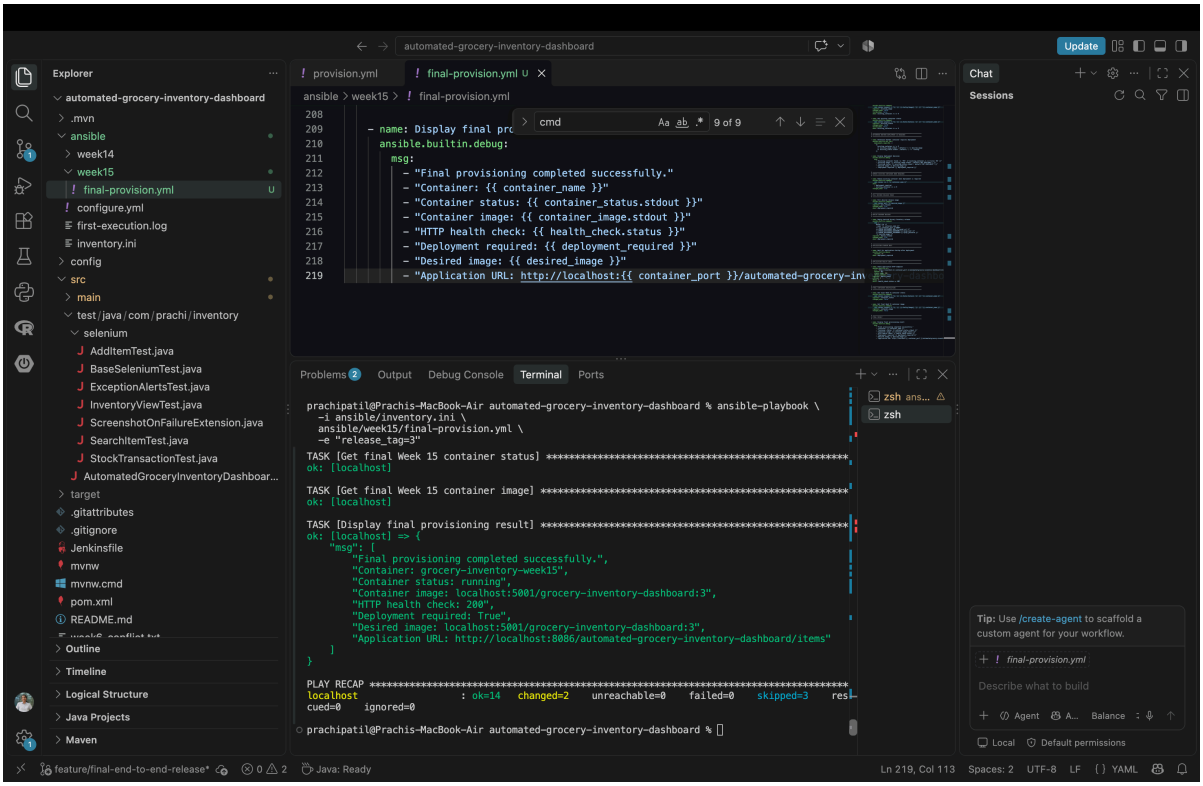


Figure 3: Selenium quality gate showing six successful tests with zero failures and zero errors.

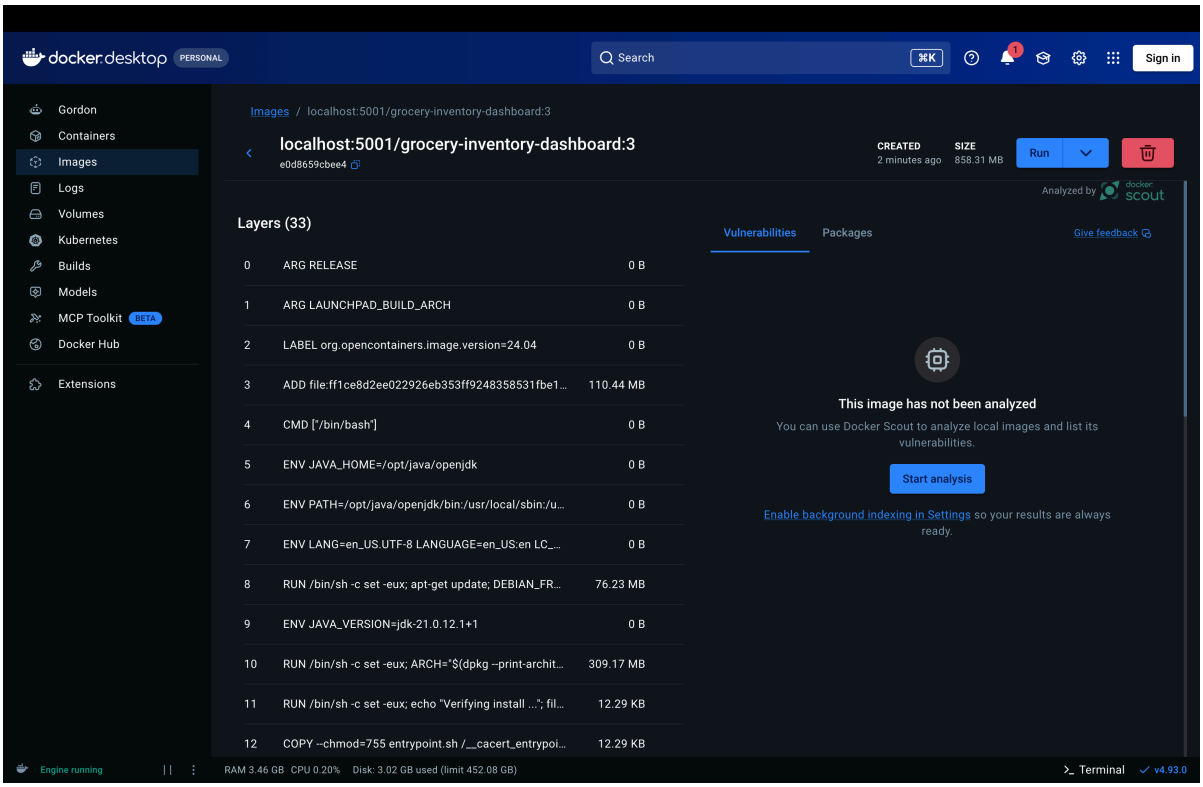


Figure 4: Docker image created for the final Jenkins build.

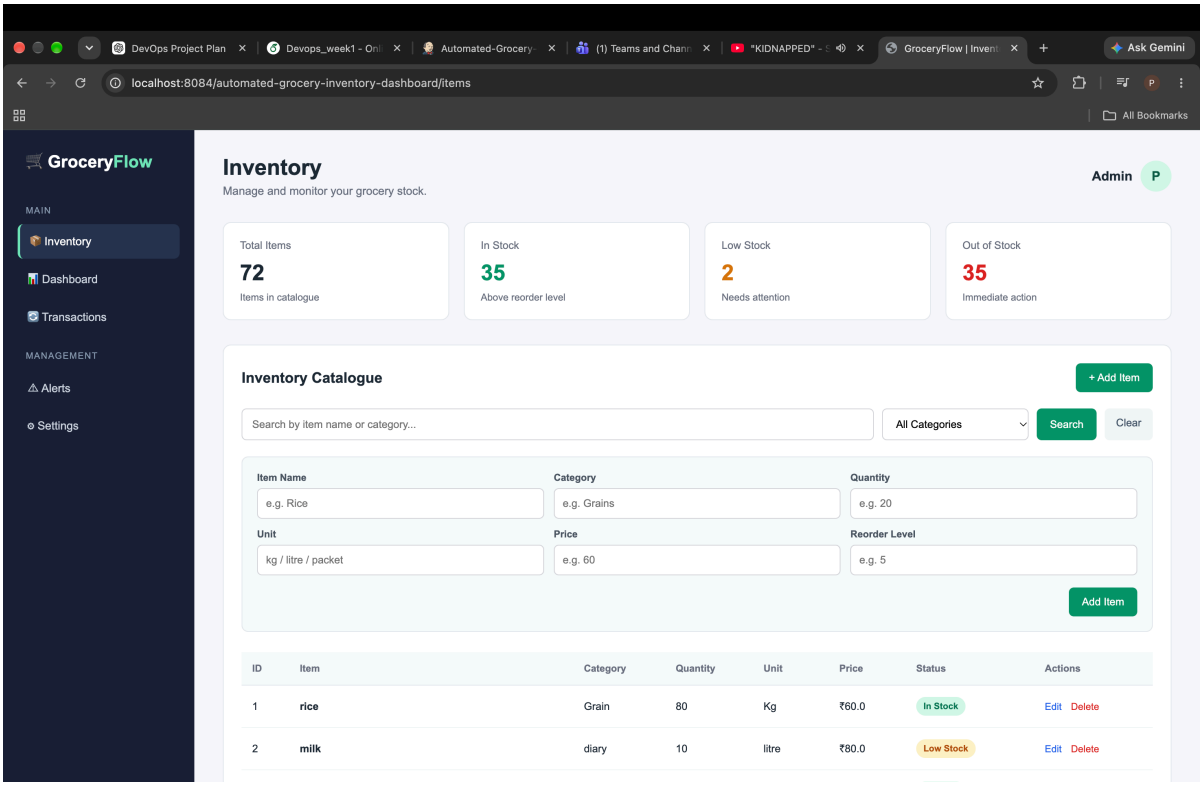


Figure 5: Final Grocery Inventory Dashboard deployed on port 8086.

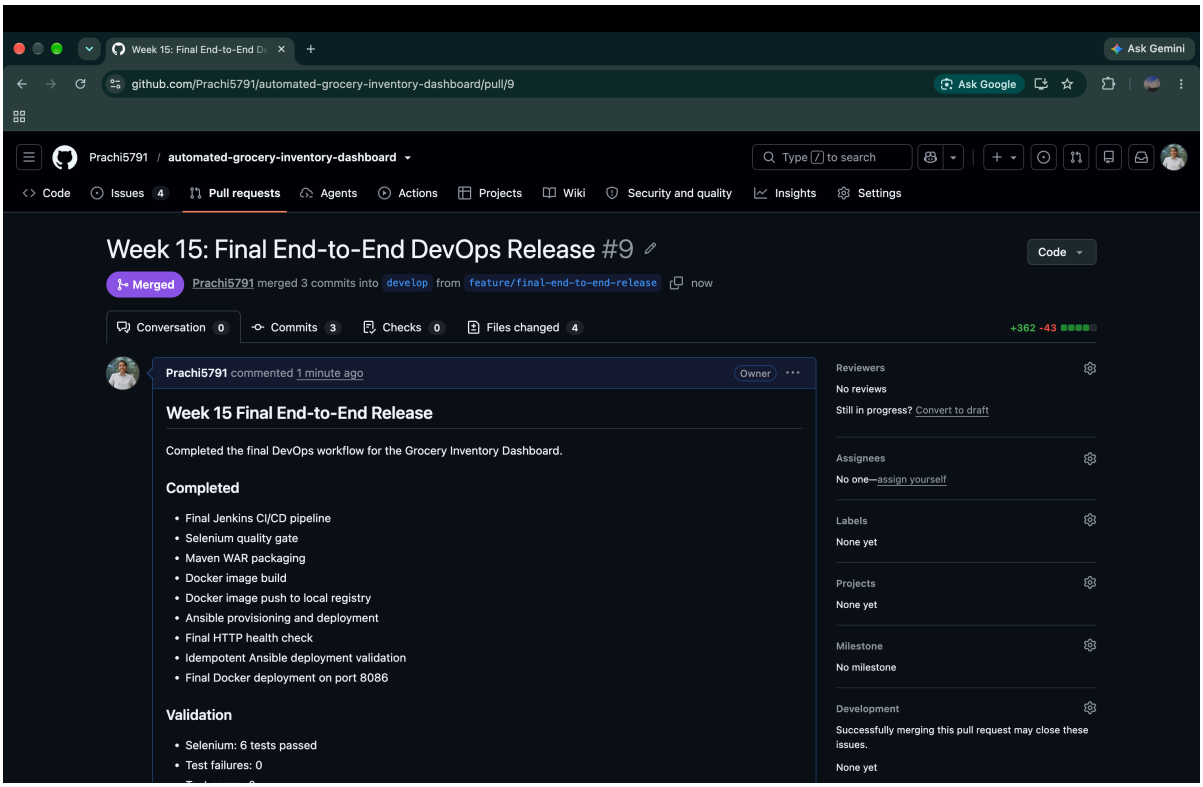


Figure 6: GitHub Pull Request #9 showing the final Week 15 release merged into develop.

21. Final Live Demonstration Flow

The final project demonstration can be performed in the following order:

1. Show the GitHub repository and the final `v1.0.0` tag.
2. Show the Jenkins final release job.
3. Open Jenkins Build #3.
4. Demonstrate the successful Selenium quality gate.
5. Show the Maven WAR packaging stage.
6. Show the Docker image build and registry push.
7. Show the Ansible provisioning output.
8. Explain the idempotency result.
9. Show the HTTP health check.
10. Open the deployed dashboard on port 8086.
11. Explain how a failed Selenium quality gate prevents later deployment stages.
12. Explain the troubleshooting performed during Week 15.

22. Conclusion

Week 15 successfully integrated the individual DevOps activities developed throughout the project into one complete end-to-end release workflow.

The final Jenkins pipeline performed source checkout, Selenium quality validation, Maven WAR packaging, Docker image construction, Docker registry push, Ansible provisioning, and final HTTP health validation.

The final Build #3 completed successfully with six Selenium tests passing and no failures or errors. The application was packaged into a Docker image tagged with the Jenkins build number and stored in the local Docker registry. Ansible verified the desired deployment state and demonstrated idempotent behaviour by making zero changes when the existing deployment already matched the desired configuration.

The final implementation was merged into the `develop` branch through Pull Request #9. The resulting release commit was tagged as `v1.0.0` and pushed to GitHub.

The project therefore demonstrates a complete DevOps lifecycle from source-code management and automated testing to containerized deployment, configuration management, reliability validation, documentation, and final release management.

A. Final Release Commands

The following commands were used to create and publish the final release:

```
git checkout develop
git pull origin develop

git tag -a v1.0.0 \
  -m "Final Week 15 End-to-End DevOps Release"

git show --stat --oneline v1.0.0

git push origin v1.0.0

git tag
```

The final tag list contains:

```
v0.6.0
v0.13.0
v0.14.0
v1.0.0
```

B. Final Project URLs

Resource	Location
GitHub Repository	https://github.com/Prachi5791/automated-grocery-inventory-dashboard
Final Application	http://localhost:8086/automated-grocery-inventory-dashboard/items
Jenkins	http://localhost:8080
Docker Registry	localhost:5001